Make std::execution's monadic operations naming scheme consistent

Document #:	P3845R0
Date:	2025-09-18
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Jonathan Müller <foonathan@jonathanmueller.dev>

Contents

§ 1 Abstract

The naming of the monadic operations in `std::execution` (`then`, `let_value`, `let_error`, `let_stopped`) are inconsistent with the naming of the monadic operations elsewhere in the standard library (`transform`, `and_then`, `or_else`). They should be renamed for greater consistency.

§ 2 Motivation and proposal

Consistency in naming is important. Once we have established a naming scheme, we should stick to it in all cases. This makes it easy for our users to guess what something is doing by looking at the name.

Right now, the monadic operations of `std::execution` are named differently than the other monadic operations in the standard library. We should remedy this inconsistency.

§ 2.1 `std::execution::then`

- `std::views::transform` operates on a range of `T` and a function `T -> U` and returns a range of `U` by applying the function to each element.
- `std::optional::transform` operates on an `optional<T>` and a function `T -> U` and returns an `optional<U>` by applying the function if the optional has a value.
- `std::expected::transform` operators on an `expected<T, E>` and a function `T -> U` and returns an `expected<U, E>` by applying the function if the expected has a value.
- `std::execution::then` operates on a sender of `T` and a function `T -> U` and returns a sender of `U` by applying the function after receiving the value.

They are all examples of the "map" operation on a monad. All expect for `std::execution::then` use the name "transform" for this operation. To be consistent, `std::execution::then` should be called `std::execution::transform`.

One argument against this change is that unlike "transform", "then" implies temporal sequencing more clearly. However, in order to transform a value, it has to be computed first, so the function of `std::execution::transform` has to be called after the value is available anyway.

Another argument against this change is that `sndr | std::execution::transform(function_returning_void)` is potentially weird because what does it mean to transform something into `void`? However, this would also apply to `std::expected::transform` which supports `void` as well.

§ 2.2 `std::execution::upon_error`, `std::execution::upon_stopped`

— `std::execution::upon_error` operates on a sender with error `E` and a function `E -> U` and returns a sender that completes successfully with value `U` if the original sender completes with error `E`.

— `std::execution::upon_stopped` operates on a sender and a function `() -> U` and returns a sender that completes successfully with value `U` if the original sender was stopped.

There is no equivalent operation in the rest of the standard library. You might think the equivalent is `std::expected::transform_error`, but unlike `upon_error` and `upon_stopped`, it does not recover from the error and only changes it. Likewise, there is no `std::execution` equivalent to `std::expected::transform_error`.

Therefore, the names `std::execution::upon_error` and `std::execution::upon_stopped` are good names and should not be changed.

§ 2.3 `std::execution::let_value`

— `std::optional::and_then` operates on an `optional<T>` and a function `T -> optional<U>` and returns an `optional<U>` by applying the function if the optional has a value.

— `std::expected::and_then` operates on an `expected<T, E>` and a function `T -> expected<U, E>` and returns an `expected<U, E>` by applying the function if the expected has a value.

— `std::execution::let_value` operates on a sender of `T` and a function `T -> sender` and returns a sender by applying the function after receiving the value.

They are all examples of the "bind" operation on a monad. All expect for `std::execution::let_value` use the name "and_then" for this operation. To be consistent, `std::execution::let_value` should be called `std::execution::and_then`.

One argument against this change is that the operation `std::execution::let_value` needs to make sure that the value received from the input sender is kept alive until the sender returned by the function completes. It does a "let x = result of sender" and allocates space for it in the operation state.

However, this is sort of implied because `sndr | let_value([&](T& x) { … })` compiles. If `T` was not kept alive, it would be easy to prevent it from compiling.

Furthermore, is the "let" part really the most important thing about this operation? Isn't the overall shape of the operation more important to convey? I'm not aware of anybody who could intuitively guess what `std::execution::let_value` does. However, if it were called `std::execution::and_then`, anybody familiar with `std::optional` and `std::expected` could guess it.

§ 2.4 `std::execution::let_error`, `std::execution::let_stopped`

— `std::optional::or_else` operates on an `optional<T>` and a function `() -> optional<T>` and returns an `optional<T>` by calling the function if the optional is empty.

— `std::expected::or_else` operates on an `expected<T, E>` and a function `E -> expected<T, F>` and returns an `expected<T, F>` by applying the function if the expected has an error.

— `std::execution::let_error` operates on a sender with error `E` and a function `E -> sender` and returns a sender by applying the function if the original sender completes with an error.

— `std::execution::let_stopped` operates on a sender and a function `() -> sender` and returns a sender by calling the function if the original sender was stopped.

They are all examples of the "bind" operation on the failure channel of a monad. All except for `std::execution::let_error/stopped` use the name "or_else" for this operation. To be consistent, `std::execution::let_error/stopped` should follow this pattern as well.

The difference being, however, that senders have two failure channels - error and stopped. The naming pattern should thus combine "error" or "stopped" with "or_else" in some way:

1. `error_or_else`, `stopped_or_else`
2. `or_error_else`, `or_stopped_else`
3. `or_else_error`, `or_else_stopped`

Of those names, `or_else_error` and `or_else_stopped` are the most clear options: They indicate that this is the "or_else" monadic operation on the "error"/"stopped" channel.

The same counter argument as in `std::execution::let_value` applies, which can be refuted in the same way. Additionally, the name `std::execution::let_stopped` is especially weird, because nothing is being kept alive for this one to begin with; the function takes no arguments.

§ 3 Before/after tables

Examples from [P2300R10].

Before

```
// The whole flow for transforming incoming requests
        into responses
sender auto snd =
    // get a sender when a new request comes
    schedule_request_start(the_read_requests_ctx)
    // make sure the request is valid; throw if not
    | let_value(validate_request)
    // process the request in a function that may be
        using a different execution resource
    | let_value(handle_request)
    // If there are errors transform them into proper
        responses
    | let_error(error_to_response)
    // If the flow is cancelled, send back a proper
        response
    | let_stopped(stopped_to_response)
    // write the result back to the client
    | let_value(send_response)
    // done
    ;
```

After

```
// The whole flow for transforming incoming requests
        into responses
sender auto snd =
    // get a sender when a new request comes
    schedule_request_start(the_read_requests_ctx)
    // make sure the request is valid; throw if not
    | and_then(validate_request)
    // process the request in a function that may be
        using a different execution resource
    | and_then(handle_request)
    // If there are errors transform them into proper
        responses
    | or_else_error(error_to_response)
    // If the flow is cancelled, send back a proper
        response
    | or_else_stopped(stopped_to_response)
    // write the result back to the client
    | and_then(send_response)
    // done
    ;
```

Here, the use of `and_then` over `let_value` is arguably clearer and makes the pipeline read more naturally.

Before

```
sender_of<dynamic_buffer> auto async_read_array(auto
        handle) {
  return just(dynamic_buffer{})
       | let_value([handle] (dynamic_buffer& buf) {
           return
         just(std::as_writeable_bytes(std::span(&buf.size,
         1)))
                | async_read(handle)
                | then(
                    [&buf] (std::size_t bytes_read) {
                      assert(bytes_read ==
         sizeof(buf.size));
                      buf.data =
         std::make_unique<std::byte[]>(buf.size);
                      return std::span(buf.data.get(),
         buf.size);
                    })
                | async_read(handle)
                | then(
                    [&buf] (std::size_t bytes_read) {
                      assert(bytes_read == buf.size);
                      return std::move(buf);
                    });
       });
}
```

After

```
sender_of<dynamic_buffer> auto async_read_array(auto
        handle) {
  return just(dynamic_buffer{})
       | and_then([handle] (dynamic_buffer& buf) {
           return
         just(std::as_writeable_bytes(std::span(&buf.size,
         1)))
                | async_read(handle)
                | transform(
                    [&buf] (std::size_t bytes_read) {
                      assert(bytes_read ==
         sizeof(buf.size));
                      buf.data =
         std::make_unique<std::byte[]>(buf.size);
                      return std::span(buf.data.get(),
         buf.size);
                    })
                | async_read(handle)
                | transform(
                    [&buf] (std::size_t bytes_read) {
                      assert(bytes_read == buf.size);
                      return std::move(buf);
                    });
       });
}
```

Here, the pattern of `just(x) | and_then([&](T& x) { … })` is slightly less clear than `just(x) | let_value([&](T& x) { … })`. The latter makes it a bit clearer that we want to ensure that `x` lives long enough for the operation. However, neither option is particularly compelling as it is still convoluted.

A better solution would be to combine it into a first-class operation: `let(x, fn)` which is equivalent to `just(x) | and_then(fn)`, or even `let(x, y, z).in(fn)` if we want to go full Haskell. That way, we also support the idiom for immovable values. This is arguably even an argument in favor of the rename to `and_then`: Having both `let` and `let_value` is potentially confusing.

§ 4 Wording

— Rename `std::execution::then` to `std::execution::transform`

— Rename `std::execution::let_value` to `std::execution::and_then`

— Rename `std::execution::let_error` to `std::execution::or_else_error`

— Rename `std::execution::let_stopped` to `std::execution::or_else_stopped`

§ 5 References

[P2300R10] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2024-06-28. `std::execution`.
https://wg21.link/p2300r10